

Primera parte

Proyecto: boutique.

lenguaje: Python.

Framework: Django.

Editor: VS code.

1 Procedimiento para crear carpeta del Proyecto: UIII_boutique_1158

2 procedimiento para abrir vs code sobre la carpeta

UIII_boutique_1158

3 procedimiento para abrir terminal en vs code

4 Procedimiento para crear carpeta entorno virtual “.venv” desde terminal de vs code

5 Procedimiento para activar el entorno virtual.

6 procedimiento para activar intérprete de python.

7 Procedimiento para instalar Django

8 procedimiento para crear proyecto backend_boutique sin duplicar carpeta.

9 procedimiento para ejecutar servidor en el puerto 8036

10 procedimiento para copiar y pegar el link en el navegador.

11 procedimiento para crear aplicacion app_boutique

12 Aqui el modelo models.py

```
--==--==--
```

```
from django.db import models
```

```
# =====
```

```
# MODELO: ventas
```

```
# =====
```

```

class Venta(models.Model):

    id_venta = models.AutoField(primary_key=True) # ID único de la venta

    total = models.DecimalField(max_digits=10, decimal_places=2) # Total de la venta

    cantidad = models.PositiveIntegerField() # Cantidad de productos vendidos

    empleado = models.CharField(max_length=100) # Nombre del empleado que realizó la
venta

    fecha = models.DateTimeField(auto_now_add=True) # Fecha y hora en que se registró
la venta


    def __str__(self):

        return f"Venta {self.id_venta} - {self.empleado} - {self.fecha}"


class Meta:

    db_table = 'ventas' # Nombre de la tabla en la base de datos

    verbose_name = 'Venta'

    verbose_name_plural = 'Ventas'

# =====

# MODELO: clientes

# =====

class Cliente(models.Model):

    id_cliente = models.AutoField(primary_key=True) # ID único de cliente

    nombre = models.CharField(max_length=100) # Nombre del cliente

    telefono = models.CharField(max_length=15, null=True, blank=True) # Teléfono, puede
ser opcional

    correo = models.EmailField(max_length=100, unique=True) # Correo electrónico del
cliente

```

```

fecha_registro = models.DateTimeField(auto_now_add=True) # Fecha de registro
direccion = models.TextField(null=True, blank=True) # Dirección del cliente
metodo_pago = models.CharField(
    max_length=50,
    choices=[ # Métodos de pago posibles
        ('tarjeta', 'Tarjeta de Crédito/Débito'),
        ('efectivo', 'Efectivo'),
        ('transferencia', 'Transferencia Bancaria'),
        ('paypal', 'PayPal')
    ],
    default='tarjeta'
) # Método de pago preferido

def __str__(self):
    return f"{self.nombre} - {self.correo}"

class Meta:
    db_table = 'clientes' # Nombre de la tabla en la base de datos
    verbose_name = 'Cliente'
    verbose_name_plural = 'Clientes'

# =====
# MODELO: Productos
# =====

class Categoria(models.Model):
    nombre = models.CharField(max_length=100)

```

```
def __str__(self):  
    return self.nombre
```

```
class Proveedor(models.Model):  
    nombre = models.CharField(max_length=100)  
    contacto = models.CharField(max_length=100, null=True, blank=True)
```

```
def __str__(self):  
    return self.nombre
```

```
class Producto(models.Model):  
    id_producto = models.AutoField(primary_key=True) # ID único de producto  
    nombre = models.CharField(max_length=150) # Nombre del producto  
    categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE,  
related_name="productos") # Relación con categoría  
    precio = models.DecimalField(max_digits=10, decimal_places=2) # Precio del producto  
    existencias = models.PositiveIntegerField() # Número de existencias disponibles  
    talla = models.CharField(max_length=10, null=True, blank=True) # Talla del producto  
(opcional)  
    proveedor = models.ForeignKey(Proveedor, on_delete=models.CASCADE,  
related_name="productos") # Relación con proveedor
```

```
def __str__(self):  
    return f"{self.nombre} - {self.categoria.nombre}"
```

```
class Meta:  
    db_table = 'productos' # Nombre de la tabla en la base de datos
```

```
verbose_name = 'Producto'
```

```
verbose_name_plural = 'Productos'
```

==--==--==--

12.5 Procedimiento para realizar las migraciones(makemigrations y migrate.

13 primero trabajamos con el MODELO: CATEGORÍA

14 En view de app_boutique crear las funciones con sus códigos correspondientes (inicio_boutique, agregar_categoria,

actualizar_categoria, realizar_actualizacion_categoria,
borrar_categoria)

15 Crear la carpeta “templates” dentro de “app_boutique”.

16 En la carpeta templates crear los archivos html (base.html, header.html, navbar.html, footer.html, inicio.html).

17 En el archivo base.html agregar bootstrap para css y js.

18 En el archivo navbar.html incluir las opciones (“Sistema de Administración boutique”, “Inicio”, “categoria”,en submenu de categorias(Aregar Categoria,ver categoria, actualizar categoria, borrar categoria), “clientes” en submenu de clientes(Aregar clientes,ver salas, actualizar salas, borrar salas)

“ventas” en submenu de ventas(Aregar ventas,ver ventas, actualizar ventas, borrar ventas), incluir iconos a las opciones principales, no en los submenu.

19 En el archivo footer.html incluir derechos de autor,fecha del sistema y “Creado por Joel Rojas, Cbtis 128” y mantenerla fija al final de la página.

- 20 En el archivo inicio.html se usa para colocar información del sistema más una imagen tomada desde la red sobre boutique.
- 21 Crear la subcarpeta carpeta categoria dentro de app_boutique\templates.
- 22 crear los archivos html con su código correspondientes de (agregar_categoria.html, ver_categorias.html mostrar en tabla con los botones ver, editar y borrar, actualizar_categoria.html, borrar_categoria.html) dentro de app_boutique\templates\categoria.
- 23 No utilizar forms.py.
- 24 procedimiento para crear el archivo urls.py en app_boutique con el código correspondiente para acceder a las funciones de views.py para operaciones de crud en categorias.
- 25 procedimiento para agregar app_boutique en settings.py de backend_boutique
- 26 realizar las configuraciones correspondiente a urls.py de backend_boutique para enlazar con app_boutique
- 27 procedimiento para registrar los modelos en admin.py y volver a realizar las migraciones.
- 27 por lo pronto solo trabajar con “categoría” dejar pendiente # MODELO: clientes y # MODELO: productos
- 28 Utilizar colores suaves, atractivos y modernos, el código de las páginas web sencillas.
- 28 No validar entrada de datos.
- 29 Al inicio crear la estructura completa de carpetas y archivos.

30 proyecto totalmente funcional.

31 finalmente ejecutar servidor en el puerto puerto 8036.